Facultad de Ingeniería
Asignatura Arquitectura y diseño de software

Taller: Escenario de evolución arquitectónica

Contexto del escenario

Después de aplicar la re-arquitectura inicial propuesta para ParkKbus Suite (monolito modular con arquitectura hexagonal, CQRS parcial y desacoplamiento de integraciones), el sistema ha mejorado significativamente su estabilidad y mantenibilidad.
Sin embargo, durante los siguientes meses de operación se presentan nuevos retos técnicos y de negocio derivados del crecimiento de la plataforma:

- El número de sedes ha aumentado de **28 a más de 60**.
- La operación diaria supera **10.000 ingresos y salidas de vehículos por día**.
- Se han integrado nuevos proveedores externos:
  - pasarelas de pago adicionales
  - sistemas municipales de movilidad
  - sensores y cámaras ANPR
- La organización desea habilitar **analítica operativa en tiempo casi real**.
- El equipo de desarrollo necesita **mayor autonomía para evolucionar partes del sistema sin afectar otras**.

Adicionalmente, la dirección tecnológica solicita que la plataforma evolucione hacia una arquitectura que:

- permita **escalar partes específicas del sistema**,
- reduzca el **impacto de fallos externos**,
- mejore la **observabilidad y monitoreo**,
- permita **desplegar componentes de forma independiente**.

Para lograrlo, el equipo de arquitectura ha identificado **cinco decisiones arquitectónicas posibles**, documentadas como **Architecture Decision Records (ADR)**.

Cada ADR propone una evolución diferente del sistema.
No todas deben implementarse al mismo tiempo.

Cada estudiante deberá:

1. Analizar las decisiones propuestas.
2. Seleccionar **una de ellas**.
3. Implementarla en la arquitectura propuesta.

ADR-01: Separación de la capa de reportes mediante CQRS

1. Contexto

En la arquitectura actual, el sistema ha mejorado su estructura interna gracias al monolito modular y la arquitectura hexagonal. Sin embargo, la generación de reportes sigue siendo una operación costosa.

Los reportes operativos y financieros realizan consultas complejas que agregan grandes volúmenes de datos, lo que puede afectar el rendimiento de las operaciones críticas del sistema, como el registro de ingreso y salida de vehículos o el procesamiento de pagos.

El problema principal es que **las operaciones de lectura intensiva (reportes y analítica)** comparten la misma base de datos y el mismo modelo de datos utilizado por las operaciones transaccionales.

Esto genera contención en la base de datos y aumenta el riesgo de degradación del sistema durante horas pico.

2. Decisión

Se decide implementar el patrón **CQRS (Command Query Responsibility Segregation)** para separar las operaciones de escritura y lectura.

En esta arquitectura:

- Las operaciones de **comando (Command)** continuarán utilizando el modelo transaccional principal.
- Las operaciones de **consulta (Query)** utilizarán un modelo optimizado para lectura, alimentado mediante eventos del sistema.

Esto permitirá que la generación de reportes y dashboards no afecte el rendimiento del sistema transaccional.

3. Justificación

CQRS es apropiado en este caso porque el sistema presenta:

- una gran diferencia entre la carga de escritura y lectura,
- consultas complejas para analítica,
- necesidad de mantener alto rendimiento en operaciones críticas.

Separar los modelos permite optimizar cada uno para su propósito específico.

Además, CQRS permite preparar el sistema para futuras capacidades de analítica en tiempo casi real.

4. Consecuencias

Beneficios

- Mejora del rendimiento de operaciones críticas.
- Mayor capacidad de escalamiento de consultas analíticas.
- Posibilidad de construir dashboards operativos en tiempo real.

Desventajas

- Mayor complejidad arquitectónica.
- Necesidad de mantener consistencia eventual entre modelos.
- Requiere infraestructura adicional para procesamiento de eventos.

5. Guía de implementación

Paso 1: Separar los casos de uso del sistema en **Commands** y **Queries** dentro de la capa de aplicación.

Ejemplo:

Commands:

- RegisterVehicleEntryCommand
- RegisterVehicleExitCommand
- ProcessPaymentCommand

Queries:

- GetParkingOccupancyQuery
- GetDailyRevenueQuery
- GetVehicleHistoryQuery

Paso 2: Crear un modelo de lectura optimizado para reportes.

Este modelo puede consistir en:

- tablas agregadas
- vistas materializadas
- read models especializados

Paso 3: Implementar un mecanismo de sincronización entre el modelo de escritura y el modelo de lectura mediante **eventos del dominio**.

Ejemplo de eventos:

- VehicleEnteredEvent
- VehicleExitedEvent
- PaymentCompletedEvent

Estos eventos actualizarán el modelo de lectura.

6. Evidencia esperada del estudiante

- diagrama actualizado mostrando la separación Command / Query
- cambios en código que separen comandos y consultas
- explicación técnica del flujo de eventos
- pruebas que demuestren que los reportes utilizan el modelo de lectura

7. Criterios de aceptación

- se separaron correctamente las operaciones de lectura y escritura

- se implementó un modelo de lectura independiente
- los reportes ya no consultan directamente el modelo transaccional
- se documentó la arquitectura CQRS

ADR-02: Introducción de mensajería asíncrona para integraciones externas

1. Contexto

El sistema ParkKbus depende de múltiples integraciones externas, incluyendo pasarelas de pago, servicios de facturación electrónica y sistemas de notificaciones.

Actualmente muchas de estas integraciones se realizan mediante llamadas síncronas. Esto significa que si uno de los sistemas externos falla o responde lentamente, el flujo principal del sistema puede verse afectado.

Esto genera un riesgo de propagación de fallos que puede impactar directamente la operación del parqueadero.

2. Decisión

Se decide introducir un sistema de mensajería asíncrona utilizando un broker de mensajes.

Las integraciones externas se realizarán mediante eventos publicados en una cola de mensajes.

El sistema principal publicará eventos y los servicios de integración los consumirán de forma independiente.

3. Justificación

El uso de mensajería permite:

- desacoplar el sistema central de los servicios externos
- mejorar la resiliencia ante fallos externos
- permitir reintentos automáticos

Además, el uso de eventos facilita la evolución hacia arquitecturas más distribuidas.

4. Consecuencias

Beneficios

- mayor resiliencia
- menor acoplamiento
- tolerancia a fallos externos

Desventajas

- complejidad operativa adicional
- necesidad de manejar consistencia eventual
- mayor esfuerzo de monitoreo

5. Guía de implementación

Paso 1: Instalar y configurar un broker de mensajes (ej. RabbitMQ).

Paso 2: Definir eventos del dominio relevantes.

Ejemplo:

- PaymentCompletedEvent
- VehicleExitedEvent
- NotificationRequestedEvent

Paso 3: Implementar productores y consumidores de eventos.

El sistema central publicará eventos y los servicios externos los consumirán.

6. Evidencia esperada del estudiante

- diagrama actualizado con broker de mensajes
- implementación de productor de eventos
- implementación de consumidor de eventos
- pruebas de procesamiento de mensajes

7. Criterios de aceptación

- las integraciones externas ya no bloquean el flujo principal
- se implementó correctamente el broker de mensajes
- los eventos del dominio están documentados

ADR-03: Extracción del servicio de notificaciones

1. Contexto

El módulo de notificaciones se encuentra integrado dentro del sistema principal.

Esto genera dependencia directa entre el flujo principal del sistema y los servicios de envío de mensajes.

2. Decisión

Se decide extraer el módulo de notificaciones como un servicio independiente.

3. Justificación

Las notificaciones:

- tienen alta variabilidad
- dependen de múltiples proveedores
- no son críticas para la transacción principal

4. Consecuencias

Beneficios

- independencia del servicio
- escalabilidad independiente

Desventajas

- mayor complejidad de infraestructura

5. Guía de implementación

Paso 1: Crear servicio NotificationService.

Paso 2: Definir API o consumidor de eventos.

Paso 3: Integrar proveedores de correo/SMS.

6. Evidencia esperada

- diagrama actualizado
- servicio independiente
- integración funcional

7. Criterios de aceptación

- servicio separado
- comunicación desacoplada
- documentación del servicio

ADR-04: Introducción de API Gateway

1. Contexto

A medida que el sistema evoluciona hacia una arquitectura más distribuida, los clientes podrían tener que interactuar con múltiples servicios.

Esto puede aumentar la complejidad del frontend y exponer detalles internos del sistema.

2. Decisión

Se decide introducir un API Gateway como punto de entrada único al sistema.

3. Justificación

El API Gateway permite:

- centralizar autenticación
- aplicar políticas de seguridad
- simplificar el acceso de clientes

4. Consecuencias

Beneficios

- punto de entrada único
- mayor control de seguridad

Desventajas

- posible cuello de botella si no se diseña bien

5. Guía de implementación

Paso 1: Definir gateway.

Paso 2: Configurar rutas hacia servicios.

Paso 3: Implementar autenticación centralizada.

6. Evidencia esperada

- diagrama actualizado
- configuración de rutas
- autenticación centralizada

7. Criterios de aceptación

- clientes consumen solo el gateway
- rutas documentadas
- seguridad implementada

ADR-05: Centralización de observabilidad y monitoreo

1. Contexto

El sistema actual utiliza logs locales en archivos dentro de las máquinas donde se ejecuta la aplicación.

Esto dificulta el monitoreo, el diagnóstico de errores y el análisis de incidentes.

2. Decisión

Implementar una plataforma centralizada de observabilidad.

3. Justificación

Permite:

- monitoreo en tiempo real
- trazabilidad de errores
- análisis de rendimiento

4. Consecuencias

Beneficios

- diagnóstico rápido
- mayor visibilidad del sistema

Desventajas

- mayor infraestructura

5. Guía de implementación

Paso 1: Configurar sistema de logging centralizado.

Paso 2: Instrumentar métricas.

Paso 3: Crear dashboards de monitoreo.

6. Evidencia esperada

- dashboards de métricas
- logs centralizados
- trazas de solicitudes

7. Criterios de aceptación

- logs centralizados funcionando
- métricas disponibles
- dashboards implementados

Resultado del taller

Cada estudiante deberá:

1. **Analizar los cinco ADR.**
2. **Elegir uno.**
3. **Diseñar e implementar la evolución arquitectónica correspondiente.**